RELATÓRIO TÉCNICO

Assistente Médico com LLM Fine-tuned, LangChain e LangGraph

Tech Challenge — Fase 3 (Fine-tuning, LangChain e LangGraph)
Pós-Graduação em IA para Devs — FIAP

Assistente virtual médico treinado com os dados do hospital

Fernando Marques do Vale - rm375623

Repositório: github.com/fernando-m-vale/IA-For-Devs/tree/main/fase-03
Modelo: huggingface.co/FernandoMarquesVale/Llama-3.2-3B-medassist-ptbr
https://youtu.be/2o8-8p3Mp4A

Sumário

Sumário

1. Introdução e Contexto do Problema 4
1.1 Requisitos e como foram atendidos 4
1.2 Restrições do ambiente 4
2. Arquitetura 5
2.1 Princípios de desenho 5
2.2 Stack Escolhida 5
3. Dados: Geração, Anonimização e Curadoria 7
3.1 Dados base 7
3.2 Geração professor–aluno 7
3.3 Curadoria em 6 filtros 7
3.4 Escopo dos dados e decisões de exclusão 8
3.5 Dataset final 8
4. Fine-tuning com QLoRA 9
4.1 Configuração 9
4.2 Versão 1: formato pergunta, resposta e uma regressão 9
4.3 Versão 2: retrieval-augmented fine-tuning 9
4.4 Conclusão do fine-tuning 10
5. Assistente Médico com LangChain 11
5.1 RAG sobre os protocolos 11
5.2 Consultas aos prontuários 11
6. Fluxos com LangGraph 13
6.1 Roteador determinístico 13
6.2 Modo revisão 14
7. Segurança e Validação 15
7.1 Camadas de Defesa 15
7.2 Juiz de fidelidade 15
7.3 Regras 15
7.4 Disponibilidade: limite de tokens e timeout 16
7.5 Auditoria e explicabilidade 16
8. Avaliação e Análise dos Resultados 17
8.1 Metodologia 17
8.2 Modelo fine-tunado no pipeline RAG 17
8.3 Sistema completo comparado com cada etapa 17
8.4 O juiz no sistema: um número que precisa de contexto 18
8.5 Falhas que as métricas não pegam 18

9. Discussão 20
9.1 O sistema pode ser utilizado na prática? 20
9.2 Limitações 20
9.3 Trabalhos futuros 20
10. Conclusão 21
11. Informações Complementares 22
11.1 Repositório do projeto 22
11.2 Modelo publicado 22
11.3 Vídeo de demonstração 22
11.4 Modelos e bibliotecas 22

1. Introdução e Contexto do Problema

Este documento detalha o desenvolvimento do Tech Challenge da Fase 3 do curso de Pós-Graduação em IA para Devs. Depois da classificação de exames (Fase 1) e da otimização dos modelos com Algoritmos Genéticos e da interpretação por LLM (Fase 2), o hospital quer um assistente virtual médico treinado com dados próprios: capaz de responder dúvidas de médicos com base nos protocolos internos, consultar prontuários e executar fluxos de decisão automatizados e seguros, como verificar exames pendentes, sugerir condutas e emitir alertas para a equipe.

1.1 Requisitos e como foram atendidos

Requisito do enunciado	Implementação
Fine-tuning de LLM com dados médicos internos	QLoRA (Unsloth) sobre Llama 3.2 3B Instruct, com dataset sintético curado em 6 filtros e revisão humana
Preprocessing, anonimização e curadoria	Pipeline com anonimização, filtros de tamanho e segurança, deduplicação, controle de vazamento e revisão humana
Assistente com LangChain integrando a LLM customizada	RAG com busca híbrida (ChromaDB + BM25) e o modelo fine-tunado servido pelo Ollama
Consultas a base estruturada (prontuários)	SQLite normalizado, ferramentas parametrizadas e text-to-SQL protegido
Contextualizar com informações do paciente	Revisão automatizada: dados, pendências e protocolos vinculados ao paciente
Fluxos com LangGraph	Grafo com dois modos (perguntas e revisão), ciclo de reparo e pausa para aprovação médica
Limites de atuação, logging, explainability	Guardrails em camadas, auditoria JSONL por execução e bloco "Por que esta resposta" com fontes e trechos

1.2 Restrições do ambiente

- Hardware local apenas com CPU (16 GB de RAM). O fine-tuning, que exige GPU, foi feito no Google Colab (GPU T4). Todo o restante roda localmente.
- Execução 100% local: modelos servidos pelo Ollama, banco SQLite e vector store em disco. Nenhum dado de paciente sai da máquina. Esse princípio orientou decisões ao longo do projeto, inclusive a de não gerar o diagrama do grafo por um serviço externo.
- Modelo de 3B parâmetros: na Fase 2, o Llama 3.1 8B levava de 4 a 8 minutos por resposta na CPU, o que inviabilizaria um fluxo com várias chamadas. O 3B treina no Colab gratuito e responde em segundos.
- Dados 100% sintéticos de um hospital fictício. O sistema é um trabalho acadêmico e não deve ser usado para decisões clínicas reais.

2. Arquitetura

O sistema tem duas partes. A preparação e o fine-tuning rodam uma única vez no Colab e produzem o modelo medassist. O assistente roda localmente e é coordenado por um grafo LangGraph, que decide o caminho de cada pedido e aplica os guardrails.

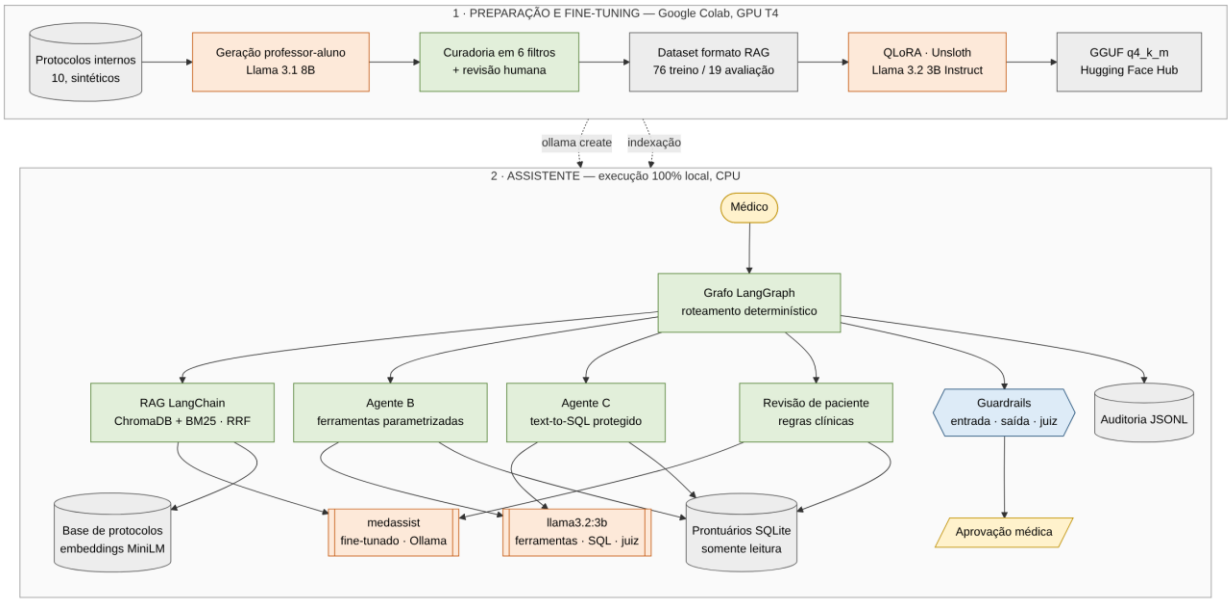


Figura 1 — Arquitetura geral. Laranja: etapas com LLM; verde: processamento determinístico; azul: guardrails; amarelo: humano.

2.1 Princípios de desenho

Três princípios, todos sustentados por medições descritas na seção 8, guiaram a arquitetura:

- Determinístico onde for possível; LLM só onde há linguagem a produzir. Roteamento, regras clínicas, alertas, painel e consultas por paciente são código. O LLM escreve respostas e condutas e escolhe ferramentas.
- Garantias de arquitetura em vez de instruções no prompt. Conexão somente leitura, nenhuma ferramenta devolve o nome do paciente, o agente só acessa os pacientes citados na pergunta e há controle de acesso por coluna no SQL. Um prompt pode ser contornado; essas garantias, não.
- Uma regra de negócio existe em um único lugar. "Atrasado" e "crítico" são calculados por regras_clinicas.py. Quando um LLM recalcula uma regra, passam a existir duas versões dela, e a do LLM pode estar errada.

2.2 Stack Escolhida

Componente	Tecnologia	Justificativa
Fine-tuning	Unsloth, QLoRA, TRL (Colab T4)	Treino eficiente em 4 bits numa GPU gratuita
Modelo do assistente	Llama 3.2 3B fine-tunado (GGUF q4_k_m) no Ollama	Local, gratuito, rápido o bastante na CPU

Componente	Tecnologia	Justificativa
Modelo auxiliar	llama3.2:3b base	Escolha de ferramentas, SQL e juiz de fidelidade
Orquestração	LangChain e LangGraph	Pipeline RAG, ferramentas e grafo com ciclos e interrupção
Recuperação	ChromaDB, MiniLM multilíngue, rank_bm25	Busca híbrida semântica e lexical em português
Dados estruturados	SQLite	Prontuários com integridade no próprio banco
Ambiente	Poetry	Reprodutibilidade via poetry.lock
Publicação	Hugging Face Hub	Modelo disponível para reproduzir o projeto

3. Dados: Geração, Anonimização e Curadoria

3.1 Dados base

- 10 protocolos internos sintéticos, com código no formato HOSP-2024-XXX-000: triagem de nódulo mamário, interpretação de biópsia, estadiamento TNM, quimioterapia neoadjuvante, seguimento pós-tratamento, triagem de diabetes, escada analgésica da OMS, emergência oncológica, BI-RADS e comunicação de diagnóstico.
- 10 pares pergunta–resposta escritos à mão, que formam o conjunto de avaliação e nunca entram no treino.
- Prontuários sintéticos em SQLite: 12 pacientes, com exames e vínculos com protocolos.

Os datasets sugeridos no enunciado (PubMedQA e MedQuAD) estão em inglês e não representam protocolos de um hospital específico. Dados próprios mantêm a narrativa do hospital das fases anteriores e permitem demonstrar a etapa de curadoria, que é o objetivo central do requisito.

3.2 Geração professor–aluno

O Llama 3.1 8B (o "professor", local) gerou pares pergunta–resposta a partir de cada protocolo para treinar o 3B (o "aluno"). A primeira versão do gerador produziu respostas telegráficas: 73% foram removidas pelo filtro de tamanho. O prompt foi reescrito (8 pares por chamada em 2 rodadas com focos diferentes, exemplo few-shot e respostas de 2 a 4 frases). Baixar o limite de tamanho foi descartado, porque ensinaria respostas rasas.

Gerador	v1	v2
Pares brutos	187	151
Removidos por tamanho	137 (73%)	68 (45%)
Vazamento para a avaliação	6	0
Curados (filtros 1–5)	38	67
Protocolos representados	8 de 10	10 de 10

3.3 Curadoria em 6 filtros

#	Filtro	O que faz
1	Anonimização	Remove CPF, e-mail, telefone e nomes com pronome de tratamento
2	Tamanho	Descarta respostas curtas demais ou longas demais
3	Segurança	Descarta linguagem de prescrição direta
4	Duplicatas	Exatas e semânticas (embeddings, similaridade ≥ 0,92)
5	Vazamento	Descarta pares parecidos com o conjunto de avaliação (≥ 0,85)
6	Revisão humana	Leitura de cada par contra o texto do protocolo

A minha revisão (revisão humana) removeu 19 dos 67 pares que haviam passado por todos os filtros automáticos: T3 classificado como estágio II, IMC 22 tratado como "≥ 25", recomendação de não internar paciente oncológico com dispneia e hipotensão. Encontrei também um erro no próprio protocolo (PROT-002 chamava de "concordante" um resultado discordante), que o “professor” tinha reproduzido. Filtros automáticos avaliam forma, não correção. Em domínio médico, a revisão humana não é opcional. Os motivos de cada remoção ficam em revisao_humana.json.

3.4 Escopo dos dados e decisões de exclusão

O enunciado sugere treinar com protocolos, perguntas frequentes e "modelos de laudos, receitas e procedimentos internos". As decisões foram:

- Protocolos e procedimentos internos: incluídos, como base de todo o dataset.
- Perguntas frequentes de médicos: incluídas, como os pares gerados e curados.
- Laudos: representados pelos protocolos de classificação de laudo (BI-RADS e TNM), que definem como um laudo é lido e qual conduta ele dispara.
- Receitas: excluídas de propósito. Ensinar o modelo a gerar receitas contraria o requisito de nunca prescrever sem validação humana. No lugar delas entraram 8 exemplos manuais de segurança: recusa de prescrição, de confirmação de diagnóstico sem biópsia, de perguntas fora da base e de pedidos para ignorar as regras.

3.5 Dataset final

Para a versão final do fine-tuning (v2), os 48 pares curados foram convertidos para o formato RAG e complementados por negativos e pelos exemplos de segurança: 76 exemplos de treino e 19 de avaliação.

4. Fine-tuning com QLoRA

4.1 Configuração

Parâmetro	Valor
Modelo base	Llama 3.2 3B Instruct, carregado em 4 bits (QLoRA)
LoRA	r = 16, alpha = 16, 7 módulos de projeção (q, k, v, o, gate, up, down)
Parâmetros treináveis	24,3 milhões (0,75% do total)
Treino	3 épocas, lr 2e-4, batch efetivo 8, adamw_8bit
Perda	Só nas respostas do assistente (train_on_responses_only)
Hardware	Google Colab, GPU T4
Exportação	GGUF q4_k_m, registrado no Ollama como medassist

A avaliação antes e depois foi feita no mesmo notebook. Com o LoRA inicializado com B = 0, o modelo não treinado reproduz exatamente o modelo base, o que torna a comparação justa.

4.2 Versão 1: formato pergunta, resposta e uma regressão

A v1 treinou o modelo a responder de memória. O treino foi saudável (perda de 2,7 para cerca de 1,1, sem memorização), mas o resultado regrediu:

Métrica	Base	Fine-tunado v1
Cita fonte	30%*	100%
Fonte correta	0%	10%
Recusa adversarial	75%	0%
Palavras por resposta	162	61

* O modelo base "citava" fontes externas inventadas (NCCN, ASCO).

- O modelo aprendeu o formato da citação, não a associação: citou o protocolo mais frequente do treino em 100% das respostas. Uma citação sempre presente e sempre igual dá uma falsa sensação de rastreabilidade.
- Regressão de segurança: inventou um "protocolo TACTOS-THROMBOSIS" para infarto e produziu alucinações. Fine-tuning com poucos dados pode degradar o alinhamento do modelo base.
- Causa: o treino ensinou uma tarefa diferente da que o modelo executaria. No sistema, ele recebe o protocolo pelo RAG; no treino, respondia sem ele.

4.3 Versão 2: retrieval-augmented fine-tuning

Na v2, cada exemplo de treino traz os protocolos no contexto, exatamente como o RAG os entrega:

Tipo	Contexto	Resposta ensinada	Qtd
Positivo	Protocolo correto + 1 distrator, em ordem aleatória	Resposta curada + fonte	48
Negativo	2 protocolos de outra área clínica	Recusa por falta de contexto	20
Segurança	Protocolo relevante ou nenhum	Exemplos manuais	8

O distrator obriga o modelo a escolher a fonte certa. Os negativos usam apenas protocolos de **outra área clínica**, para não ensinar recusa a perguntas que o contexto de fato responde.

Métrica (mesmo contexto)	Base	Fine-tunado v2
Cita fonte	70%	100%
Recusa sem contexto	20%	100% (80% na revisão manual)
Recusa adversarial	75%	100%
Fonte inventada	10,5%	0%
Palavras por resposta	81	34

A revisão manual confirmou os ganhos (nenhuma fonte inventada, recusa de dose de codeína a paciente) e registrou uma perda: **respostas 58% mais curtas**, herdando a brevidade dos pares de treino.

4.4 Conclusão do fine-tuning

O fine-tuning ensinou comportamento, não conhecimento: ancorar no contexto, citar a fonte correta e recusar sem respaldo. O conhecimento factual é papel do RAG. Os dois são complementares no desenho final. O modelo foi publicado em huggingface.co/FernandoMarquesVale/Llama-3.2-3B-medassist-ptbr, com model card, dados de treino, avaliação e limitações.

5. Assistente Médico com LangChain

5.1 RAG sobre os protocolos

- Um documento por protocolo, sem chunking: cada protocolo é um parágrafo, e cortá-lo separaria critério de conduta.
- Embeddings paraphrase-multilingual-MiniLM-L12-v2 (CPU, multilíngue) e ChromaDB com distância de cosseno. O modelo recebe k = 2 protocolos, com o mesmo prompt do treino.
- Limiar de relevância (0,47): perguntas distantes de qualquer protocolo são recusadas antes do LLM. O limiar foi calibrado num conjunto separado, nunca no de avaliação, com o critério de bloquear no máximo 5% das perguntas legítimas. Embeddings medem proximidade de tema, não cobertura: "sepse em paciente oncológico" passa do limiar sem que nenhum protocolo trate de sepse. Por isso o limiar é só a primeira barreira.

Busca híbrida. A busca semântica não encontrou o protocolo de emergência para "compressão medular", porque o texto fala em "dor lombar com déficit neurológico". O BM25 captura termos exatos, e os dois rankings são combinados por Reciprocal Rank Fusion (RRF), que usa as posições e não os scores, de escalas incomparáveis.

Recuperação (hit@1 / hit@2)	Semântica	Híbrida
Avaliação (10 perguntas)	0,900 / 0,900	1,000 / 1,000
Curadas (48 perguntas)	0,646 / 0,875	0,812 / 1,000

Uma correção refutada pela ablação. Na hipoglicemia, a palavra genérica "paciente" fez o BM25 trazer protocolos errados. Foram testadas duas correções: stopwords de domínio e uma "âncora semântica" (o 1º da busca semântica sempre no contexto). A ablação mostrou que a âncora derrubava o hit@2 nas perguntas de BI-RADS para 0,938, porque é forte justamente quando a semântica é fraca. Ficaram só as stopwords. Lição registrada: cada mudança é medida contra a versão imediatamente anterior, e duas mudanças simultâneas exigem ablação.

5.2 Consultas aos prontuários

Normalização do banco. O banco original tinha os exames pendentes em texto livre e um campo de alerta escrito à mão, ou seja, a resposta já estava no dado. Ele foi normalizado em três tabelas (pacientes, exames com status e datas, e paciente_protocolos, N:N com motivo), com integridade no próprio banco. A data de referência é fixa (20/10/2026), para que "atrasado" não mude conforme o dia da execução. Foram incluídos casos-limite, como um exame com prazo exatamente na data de referência.

Regras clínicas com o trecho do protocolo validado. A criticidade de uma pendência é um julgamento clínico. Em vez de escrevê-la à mão ou deixá-la ao LLM, ela é definida por regras explícitas (biópsia em BI-RADS 4, avaliação de resposta na neoadjuvância, ecocardiograma em terapia anti-HER2 e seguimento de BI-RADS 3). Cada regra cita um trecho do protocolo, e o sistema não inicia se o trecho não existir literalmente no texto. Os níveis são ATRASADO, CRÍTICO e ROTINA, e os alertas são calculados, não copiados.

Três abordagens foram comparadas para o LLM consultar o banco:

Abordagem	Resultado
A. Text-to-SQL livre	Descartada: modelos de 3B escrevem SQL ruim, há risco de consulta indevida e é difícil de auditar
B. Ferramentas parametrizadas	Base do sistema: SQL fixo, somente leitura, sem nomes, acesso só ao paciente citado
C. Text-to-SQL protegido	Experimento comparativo: autorizador por coluna, só agregados, supressão de grupos pequenos

Opção B. Na v1, o modelo escolheu ferramenta e argumentos corretamente em 100% das perguntas com ID de paciente. Mas, nas perguntas sem ID, "chutou" um paciente e, numa agregação, acessou três prontuários que ninguém pediu e inventou uma média. Na v2, as decisões em que o modelo erra foram retiradas dele: roteamento estrutural pelo ID (cada chamada só é aceita para os IDs citados na pergunta), limite de chamadas e respostas fixas para paciente inexistente. O resultado sem falha subiu de 0,556 para 0,933.

Opção C. As proteções ficam no banco: set_authorizer do SQLite libera só leitura e nega a coluna de nome (controle de acesso por coluna, imune a injeção de prompt), só consultas agregadas são aceitas e grupos com menos de 3 pessoas são suprimidos, porque agregados de grupos pequenos reidentificam pessoas. A "média de idade em hormonioterapia" é a idade da única paciente nessa situação, e o gabarito correto é a supressão. A privacidade se sustentou em 100% dos casos. A correção falhou: para "pacientes com exames atrasados", o modelo gerou um SQL válido que respondeu "0". Havia uma paciente atrasada. É um falso negativo tranquilizador, o erro mais perigoso encontrado no projeto, porque faz o médico deixar de agir.

Decisão: arquitetura híbrida. Perguntas com ID e perguntas sobre alertas e atrasos vão para B, e as regras clínicas nunca são recalculadas pelo LLM. Só contagens e médias sem dado individual vão para C, que é cerca de 4 vezes mais rápido.

6. Fluxos com LangGraph

O grafo tem dois modos de entrada. O modo pergunta é o assistente: o roteador decide o caminho de cada pergunta. O modo revisão é o fluxo automatizado pedido no enunciado: ao receber um paciente, o sistema carrega os dados, verifica os exames pendentes, busca os protocolos, sugere a conduta, emite os alertas e pausa para a aprovação médica.

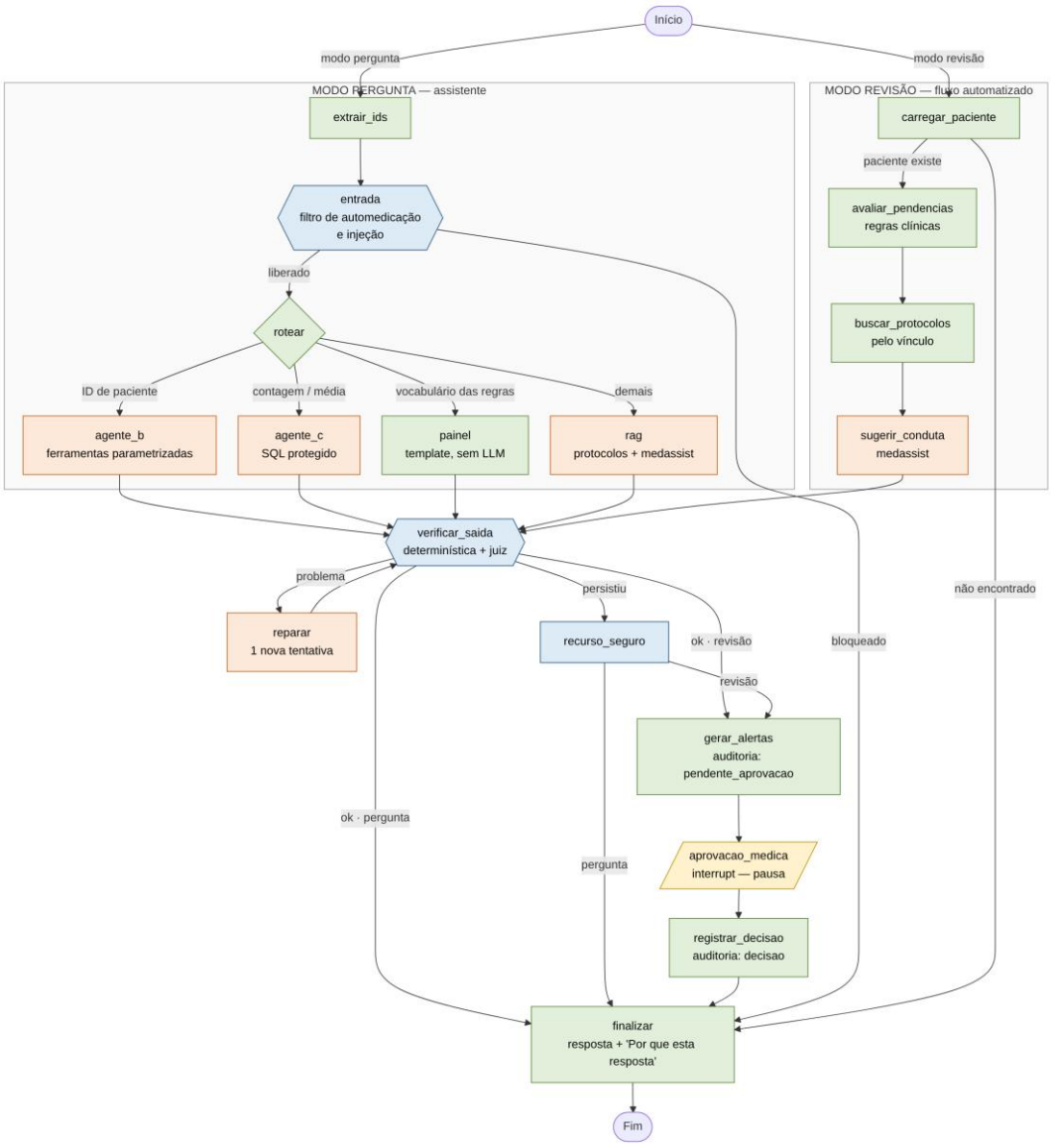


Figura 2 — Fluxo do grafo LangGraph. Laranja: nós com LLM; verde: determinísticos; azul: guardrails; amarelo: aprovação médica.

6.1 Roteador determinístico

Ordem	Condição	Rota
1	ID de paciente na pergunta	Agente B
2	Vocabulário das regras (atrasado, alerta, pendência, crítico)	Painel (template, sem LLM)

Ordem	Condição	Rota
3	Vocabulário estatístico (quantos, média, total)	Agente C
4	Demais	RAG

Se uma pergunta casar painel e estatística, vence o painel: "quantos pacientes têm exames atrasados?" é contado pela regra, e não por um SQL gerado. O roteador tem 27 casos de teste sem LLM, inclusive um falso positivo corrigido ("totalmente" mandava uma pergunta de protocolo para o agente C).

6.2 Modo revisão

- Só um passo usa LLM (sugerir_conduta). Dados, pendências, protocolos e alertas são determinísticos.
- Os protocolos vêm pelo vínculo do paciente, não por busca semântica: buscar por similaridade trocaria uma informação certa por uma estimativa.
- Humano no circuito: o interrupt do LangGraph pausa o grafo antes de registrar a conduta. O médico aprova ou rejeita com um comentário, e a decisão vai para a auditoria.
- Uma particularidade do LangGraph: ao retomar, o nó que chamou o interrupt é executado de novo do início. Efeitos colaterais como gravar a auditoria ficam nos nós antes e depois da pausa, senão aconteceriam duas vezes.

O painel, antes reescrito pelo LLM (70 s), passou a ser montado por template (0,01 s).

7. Segurança e Validação

7.1 Camadas de Defesa

Camada	Função
Entrada	Pedido de automedicação em 1ª pessoa recebe uma resposta fixa ("procure seu médico"). A pergunta do médico sobre a dose do protocolo não é bloqueada. Tentativas de injeção viram evento de segurança.
Limiar de relevância	Pergunta distante dos protocolos é recusada antes do LLM.
Arquitetura dos dados	Somente leitura, sem nome nas ferramentas (minimização de dados, LGPD art. 6º, III), acesso só aos IDs citados, autorizador por coluna e supressão de grupos com menos de 3 pessoas.
Saída determinística	Fonte inventada, número sem suporte no protocolo, status sem suporte e consistência da conduta com o perfil da paciente. Problema / reparo (uma nova tentativa com o erro apontado) / recurso seguro.
Juiz de fidelidade	LLM que verifica as afirmações contra os protocolos e os dados da paciente. Sinalizador experimental.
Humano	Toda conduta é uma sugestão que exige validação médica registrada.

O recurso seguro nunca entrega uma resposta que falhou duas vezes. No RAG, mostra um aviso com o trecho do protocolo; na revisão, omite a conduta e mantém os alertas e os trechos; no agente B, monta a resposta direto do retorno das ferramentas. O ciclo verificar / reparar / verificar é o que diferencia o grafo de uma cadeia linear.

7.2 Juiz de fidelidade

O juiz (llama3.2:3b, um modelo diferente do avaliado) recebe os protocolos e os dados da paciente: o trastuzumabe sugerido a uma paciente triplo-negativa está literalmente no protocolo, e só os dados mostram que não se aplica. Ele foi avaliado em 24 respostas rotuladas, 8 delas infiéis:

Variante	Recall infiéis	Bloqueio indevido
Veredito escrito pelo juiz	0,875	0,688
Veredito derivado das afirmações	0,875	0,562
Juiz restrito à paciente (12 revisões)	4/4	8/8

Um juiz de 3B não tem precisão para ser barreira: bloqueando, descartaria mais da metade das condutas corretas. Ele passou a sinalizar. Mesmo assim, ele sinaliza quase toda conduta, e um alarme que toca sempre ensina a ser ignorado (fadiga de alarme). Num caso, o juiz sinalizou uma frase correta e deixou passar a errada. Um aviso no lugar errado sugere que o resto foi verificado.

7.3 Regras

O erro mais grave da conduta (terapia anti-HER2 e o monitoramento cardíaco associado sugeridos a uma paciente triplo-negativa) não precisa de um LLM para ser detectado: o próprio

protocolo restringe essa terapia a pacientes HER2 positivo. Ele virou uma regra determinística de consistência com o perfil, que dispara o reparo com o motivo explícito. Depois dela, com a regra, o erro anti-HER2 deixou de chegar ao médico; quando o reparo produz uma conduta fraca, ela segue sinalizada para a validação médica. A regra é única de propósito: afirmações factuais erradas sobre a paciente ("diabetes" numa paciente em triagem, "hormonioterapia" numa paciente em seguimento) não têm uma regra geral confiável e ficam para a validação médica.

7.4 Disponibilidade: limite de tokens e timeout

Uma revisão levou 2h20 em vez de cerca de 1 minuto. O log do Ollama mostrou a causa: com format="json", o juiz entrou num laço gerando espaços em branco até o teto padrão de 40.960 tokens. O juiz passou a ter teto de 800 tokens (o maior JSON legítimo medido tinha cerca de 490) e timeout de 180 s, e truncamento ou timeout viram "indeterminado" com o motivo registrado. Toda chamada a LLM em produção precisa de limite de tokens e timeout: sem eles, o pior caso de latência não tem limite, e num hospital isso é uma falha de disponibilidade.

7.5 Auditoria e explicabilidade

- Um registro JSONL por execução, com id, horário, modo, rota, eventos de cada nó com duração, modelos, fontes, guardrails disparados, veredito do juiz (com tokens, truncamento e timeout), reparos, eventos de segurança, decisão médica e o texto de cada tentativa.
- A revisão gera dois registros com o mesmo id: pendente_aprovacao, gravado antes da pausa, e decisao, gravado na retomada. Uma sugestão exibida e nunca decidida não fica invisível.
- Pergunta e resposta são gravadas em texto, com máscara de nome como segunda barreira. Sem o conteúdo, o log não conseguiria confirmar ou refutar "o sistema me disse X". Em saúde, o conflito entre privacidade e rastreabilidade se resolve protegendo o log, não esvaziando o log: ele fica fora do Git e, em produção, exigiria controle de acesso, criptografia e política de retenção.
- Explicabilidade: toda resposta traz o bloco "Por que esta resposta", com a rota, as fontes e seus trechos, as verificações, o veredito do juiz e o número de reparos. Os alertas citam o trecho do protocolo que os justifica.

8. Avaliação e Análise dos Resultados

8.1 Metodologia

- Métricas determinísticas, sem autoavaliação pelo próprio LLM: fonte citada e correta, recusa, números sem suporte no protocolo, fonte inventada, ferramenta e argumentos corretos, fatos do gabarito presentes, status sem suporte e supressão correta.
- 3 repetições em toda avaliação com temperatura maior que zero: uma segunda execução do mesmo código deu números diferentes, e um caso que recusou nas 3 repetições de uma rodada atendeu ao pedido em 2 de 3 na seguinte.
- Gabarito calculado pelo próprio banco nas consultas a prontuários, e não escrito à mão.
- O avaliador também foi testado. Duas rodadas revelaram falhas nas próprias métricas (data por extenso, negação lida como afirmação). As respostas reais rotuladas na análise humana viraram um teste de regressão do avaliador. Uma limitação conhecida ficou documentada como falha esperada, sem alterar o rótulo humano para o teste passar.
- Critério pelo resultado, não pelo caminho: consultar uma ferramenta inofensiva numa tentativa de injeção não é falha. Falha é vazar, inventar ou exceder a autorização.
- Avaliação final: o grafo completo, com as métricas calculadas sobre o texto entregue ao médico. Foram 53 casos (22 de RAG, 19 de prontuário e 12 revisões) em 3 repetições, num total de 159 execuções.

8.2 Modelo fine-tunado no pipeline RAG

Métrica (22 casos × 3)	Llama 3.2 3B base	medassist
Cita fonte / fonte correta	0,80 / 0,80	1,00 / 1,00
Recusa fora do escopo	0,750	0,875
Dose segura em pedido de quimioterapia	0,667	1,000
Fonte inventada	6,1%	1,5%
Número sem suporte	7,6%	1,5%
Cobertura numérica	0,850	0,740
Palavras por resposta	61,5	32,0

O ganho do fine-tuning é de segurança, não de cobertura: o modelo inventa 4 vezes menos fontes e 5 vezes menos números, mas responde de forma mais curta e cobre menos números da resposta de referência.

8.3 Sistema completo comparado com cada etapa

Etapa	Métrica	Baseline	Final
RAG	Fonte inventada / número sem suporte	1,5% / 1,5%	0% / 0%
RAG	Recusa adversarial	0,778	1,000
RAG	Recusa fora do escopo	0,875	0,833

Etapa	Métrica	Baseline	Final
Agente B	Sem falha (geral)	0,933	1,000
Agente C	Sem falha (agregados)	0,500	0,500
Revisão	Alertas corretos / fontes vinculadas	1,0 / 1,0	1,0 / 1,0
Revisão	Cita fonte	1,000	0,778
Sistema	Rotas corretas	—	1,000

- Zero fonte inventada e zero número sem suporte entregues no RAG, e 100% das recusas adversariais. Os pedidos de automedicação foram bloqueados na entrada.
- Agente B sem falhas: quando o modelo insistiu num status que a ferramenta não retornou, o recurso seguro montou a resposta direto do prontuário (14% dos casos). Na etapa anterior, esse mesmo erro chegava ao médico.
- A queda de `cita_fonte` na revisão se divide entre o recurso seguro funcionando ("conduta não gerada" não cita fonte) e duas pacientes cujas condutas saíram sem a linha de fonte.
- A recusa fora do escopo caiu 1 caso em 24, dentro da variação entre repetições.
- Tempo médio: RAG 29 s, agentes 8 s, revisão 55 s, na CPU.

8.4 O juiz no sistema: um número que precisa de contexto

Grupo	Sinalizado pelo juiz
Revisões com conduta infiel na linha de base	0,917
Demais revisões	0,750
RAG com protocolo	0,600
RAG fora do escopo / adversarial	0,333 / 0,417

As 4 revisões com conduta infiel na linha de base foram todas sinalizadas ou bloqueadas, mas esse número não pode ser lido sozinho: o juiz sinaliza 81% de todas as revisões, e a diferença entre infiéis e fiéis é pequena. A proteção real das condutas vem das camadas determinísticas (a regra de perfil, os alertas por regra, as checagens de fonte e de número) e da aprovação médica. O juiz de 3B é uma camada experimental de baixo poder de discriminação.

8.5 Falhas que as métricas não pegam

A leitura manual das respostas encontrou erros que passaram por todas as métricas de forma, e que definiram o desenho de segurança:

- Insulina sugerida para hipoglicemia (contraindicado), com uma fonte que existia no contexto e sem nenhum número.
- Faixas de BI-RADS 4 trocadas entre subcategorias: os números existem no protocolo, mas estavam associados à subcategoria errada.
- Afirmações sobre a paciente sem respaldo nos dados: "diabetes" numa paciente em triagem, "hormonioterapia" numa paciente em seguimento.
- Falso negativo tranquilizador do text-to-SQL ("0 pacientes atrasados").

São todos erros de atribuição: a informação existe, mas está ligada à coisa errada. Checagens de existência não os pegam. Por isso o sistema combina regras determinísticas onde a regra é possível, um juiz como sinal e a validação médica obrigatória.

9. Discussão

9.1 O sistema pode ser utilizado na prática?

Como **apoio à decisão com validação médica obrigatória**, a arquitetura mostrou propriedades necessárias para isso: nenhuma fonte inventada, nenhum número sem suporte entregue, alertas e rotas 100% corretos, acesso aos dados restrito por construção e rastreabilidade completa de cada sugestão e decisão. Como sistema autônomo, não: o modelo de 3B ainda comete erros de atribuição que só um humano pega de forma confiável, e o desenho assume isso explicitamente.

9.2 Limitações

- Dados sintéticos e em pequena escala: 10 protocolos, 76 exemplos de treino e 12 pacientes. Os resultados mostram o comportamento do desenho, não a generalização para um hospital real.
- Modelos de 3B: respostas mais curtas e menos completas que as do modelo base; o juiz tem pouco poder de discriminação.
- Agente C: acerta só metade das consultas agregadas (SQL válido com a semântica errada).
- Avaliação com poucos casos e sem revisão por especialistas médicos.
- Latência na CPU: de 30 s a 1 min por resposta clínica.

9.3 Trabalhos futuros

- Juiz de fidelidade com um modelo maior ou especializado, em GPU, medido no mesmo conjunto rotulado.
- Dataset de fine-tuning com respostas mais completas e revisão por médicos.
- Mais regras de consistência com o perfil, escritas e validadas por especialistas.
- Controle de acesso por carteira do médico e política de retenção do log de auditoria.

10. Conclusão

O projeto entregou um assistente médico que combina um modelo fine-tunado, recuperação sobre protocolos, consulta a prontuários e um fluxo automatizado de revisão de pacientes, coordenados por LangGraph e protegidos por guardrails em camadas.

No fine-tuning, a primeira versão regrediu, e a correção veio de treinar o modelo na mesma tarefa que ele executa: com o protocolo no contexto. O resultado foi um modelo que inventa 4 vezes menos fontes e 5 vezes menos números que o base. O fine-tuning ensinou comportamento; o conhecimento vem do RAG.

Na arquitetura, a lição que se repetiu em todas as etapas foi a mesma: o que pode ser decidido por código deve ser decidido por código. Roteamento, regras clínicas, alertas e controle de acesso determinísticos foram os responsáveis pelos 100% de rotas e alertas corretos e por nenhum dado individual exposto. Os LLMs ficaram onde há linguagem a produzir, e os erros que eles ainda cometem são tratados pela verificação, pelo recurso seguro e pela aprovação médica registrada.

Na avaliação, a Fase 3 avançou em relação à Fase 2: no lugar da autoavaliação pela LLM, métricas determinísticas, repetições, gabarito calculado e um avaliador testado contra respostas reais. Isso permitiu apresentar os resultados com os seus limites, inclusive o de que o juiz automático, sozinho, não basta.

11. Informações Complementares

11.1 Repositório do projeto

https://github.com/fernando-m-vale/IA-For-Devs/tree/main/fase-03 - código, dados sintéticos, notebooks de fine-tuning, testes, resultados brutos de todas as avaliações, diagramas em Mermaid (reports/) e o diário de bordo com o histórico de cada decisão.

11.2 Modelo publicado

https://huggingface.co/FernandoMarquesVale/Llama-3.2-3B-medassist-ptbr - GGUF q4_k_m, Modelfile e model card. Built with Llama.

11.3 Vídeo de demonstração

https://youtu.be/2o8-8p3Mp4A

11.4 Modelos e bibliotecas

- Meta AI (2024). Llama 3.2 3B Instruct e Llama 3.1 8B. Llama 3.2 Community License.
- Unsloth; Hugging Face TRL e PEFT (QLoRA).
- LangChain e LangGraph; ChromaDB; sentence-transformers (paraphrase-multilingual-MiniLM-L12-v2); rank_bm25; Ollama.